

# Neural Successive Cancellation Decoding of Polar Codes for the Two-User Gaussian Multiple Access Channel

## A Comprehensive Technical Report

**Project:** EE Master's Thesis **Date:** April 3, 2026 **Repository:** <https://github.com/eitanspi/polar-codes-mac>

---

## 1. Introduction

This report documents the development of a neural network-based successive cancellation (SC) decoder for polar codes on the two-user Gaussian Multiple Access Channel (MAC). The project achieves neural decoding performance that matches the analytical SC decoder at block lengths  $N \leq 128$ , but encounters a persistent performance gap at  $N \geq 256$ . This report explains the system in full detail, documents every approach that was tried (both successful and failed), and characterizes the remaining gap to guide future research.

### 1.1 Goal

Replace ALL analytical operations in the SC MAC decoder with learned neural networks, enabling deployment on channels where analytical transition probabilities are unknown or too complex to compute. The neural decoder should match the analytical SC decoder's block error rate (BLER) at all practical code lengths.

### 1.2 Summary of Results

| N   | NN-SC BLER | SC BLER | Ratio | Status            |
|-----|------------|---------|-------|-------------------|
| 32  | 0.046      | 0.046   | 1.0x  | <b>Matches SC</b> |
| 64  | 0.026      | 0.025   | 1.03x | <b>Matches SC</b> |
| 128 | 0.017      | 0.016   | 1.04x | <b>Matches SC</b> |
| 256 | 0.015      | 0.005   | 3.0x  | Gap remains       |
| 512 | 0.012      | 0.001   | 12x   | Large gap         |

With CRC-aided Neural SCL (list decoding + CRC check),  $N=128$  achieves  $\text{BLER}=0.002$ , beating even analytical SCL( $L=4$ ) at 0.004.

---

## 2. Channel Model

### 2.1 Gaussian MAC (GMAC)

Two independent users transmit binary messages  $U, V$  in  $\{0,1\}^N$  through a shared Gaussian channel:

$$Z[i] = (1 - 2 \cdot X[i]) + (1 - 2 \cdot Y[i]) + W[i]$$

where  $X = \text{polar\_encode}(U)$ ,  $Y = \text{polar\_encode}(V)$  are BPSK-modulated codewords, and  $W \sim N(0, \sigma^2)$  is additive white Gaussian noise. The receiver observes  $Z$  and must decode both  $U$  and  $V$ .

**At SNR = 6 dB ( $\sigma^2 = 0.251$ ):** -  $I(Z;X) = 0.464$  (U marginal capacity) -  $I(Z;Y|X) = 0.912$  (V conditional capacity) -  $I(Z;X,Y) = 1.376$  (sum capacity)

The U marginal channel is a **Gaussian mixture** (not simple AWGN), because  $Y$  is unknown when decoding  $U$ . This makes GMAC significantly harder than single-user channels.

## 2.2 Other Channels (for context)

- **BEMAC** (Binary Erasure MAC):  $Z = X + Y$  in  $\{0,1,2\}$ . Discrete output. The neural decoder matches/beats SC at ALL N including 1024 on this channel.
- **ABNMAC** (Additive Binary Noise MAC):  $Z = (XXORE\_x, YXORE\_y)$ . Also discrete.

The GMAC is the hardest channel because of the continuous output and Gaussian mixture marginal.

## 3. Polar Code Construction

### 3.1 Encoder

The polar encoder applies  $X = U \cdot B\_N \cdot F^{\otimes n} \pmod{2}$ , where  $B\_N$  is the bit-reversal permutation and  $F = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ . Implementation uses the butterfly structure: bit-reversal followed by  $n$  stages of XOR operations. Complexity:  $O(N \log N)$ . Both users encode independently.

### 3.2 Frozen Set Design

The frozen set determines which bit positions carry information vs. are fixed to 0. This is critical for performance.

**Analytical methods:** - **Bhattacharyya recursion:**  $Z^- = 2Z - Z^2$ ,  $Z^+ = Z^2$ . Works for extreme paths (Class A/C). - **Gaussian Approximation (GA):** Tracks LLR mean through polarization stages. Tighter for Gaussian channels.

**Critical finding: GA design is WRONG for Class B.** GA assumes the extreme path  $0^N 1^N$ , but Class B uses an interleaved path. Using GA design for Class B gives catastrophic BLER (70%+ instead of 5%). **Monte Carlo genie-aided design must be used for Class B.**

**Monte Carlo Design:** Run the SC decoder with true bits (genie), count errors per synthetic channel over many trials. This accounts for the actual decoding path. Pre-computed MC designs are stored as .npz files for  $N = 8$  to 2048.

### 3.3 Decoding Paths and Code Classes

The MAC SC decoder processes  $2N$  leaf positions following a path  $b$  in  $\{0,1\}^{2N}$ :

| Class | Path                          | Description             | CalcParent needed |
|-------|-------------------------------|-------------------------|-------------------|
| A     | $1^N 0^N$                     | All V first, then all U | No                |
| C     | $0^N 1^N$                     | All U first, then all V | No                |
| B     | $0^{\{N/2\}} 1^N 0^{\{N/2\}}$ | <b>Interleaved</b>      | <b>Yes</b>        |

**Class B achieves the symmetric rate point  $R_u = R_v \sim 0.48$** , which is the most challenging because CalcParent operations are needed at many tree positions, and information from both users must be combined during the tree walk.

**Important:** At the symmetric rate point,  $R_u = 0.469 > I(Z;X) = 0.464$ . User U operates ABOVE its marginal capacity — it literally cannot be decoded alone without V’s help. This is why the interleaved path is necessary and why independent per-user decoders cannot work for Class B.

---

## 4. Analytical SC Decoder

### 4.1 Tree Structure

The decoder walks a binary tree with  $2N-1$  edges. Edge 1 is the root (carries  $N$  embeddings), edges  $N$  to  $2N-1$  are leaves (carry 1 embedding each). Each edge carries a  $2 \times 2$  log-probability tensor  $P(u,v|\text{observations})$ :

```
Tensor = [[log P(u=0,v=0), log P(u=0,v=1)],
           [log P(u=1,v=0), log P(u=1,v=1)]]
```

### 4.2 Core Operations

- **CalcLeft (f-node):** Circular convolution of parent and right-child tensors. Produces the top-down message for the left child.
- **CalcRight (g-node):** Normalized element-wise product of parent and left-child tensors. Produces the top-down message for the right child, conditioned on the left child’s decision.
- **CalcParent:** Inverse operation — combines left and right children to reconstruct the parent tensor. **Only needed for interleaved paths (Class B).**

### 4.3 Sequential Tree Walk

The decoder follows the path b step by step: 1. Navigate the tree to the target leaf (CalcLeft/CalcRight/CalcParent to compute tensors along the way) 2. At the leaf: combine top-down and bottom-up tensors, extract 4-class probabilities 3. Make a hard decision  $(u,v) = \text{argmax}$  4. Set the leaf tensor to reflect the decision 5. Move to the next leaf

For  $N=256$ , this requires  $\sim 1500$  sequential tensor operations.

### 4.4 SCL Decoder

The SC List decoder maintains  $L$  candidate paths. At each non-frozen leaf, each candidate forks into 2 new candidates (one per bit value). The  $L$  best candidates (by cumulative log-probability) are kept.

**SCL( $L=4$ ) results at SNR=6dB, Class B:** -  $N=32$ : BLER=0.023 -  $N=128$ : BLER=0.004 -  $N=256$ : BLER=0.001 -  $N=512$ : BLER=0.000 (essentially perfect)

---

## 5. Neural Decoder Architecture

### 5.1 Design Philosophy

The neural decoder replaces ALL analytical tensor operations with learned MLPs. The architecture mirrors the SC decoder’s computational graph:

| Operation          | Analytical             | Neural             | Input $\rightarrow$ Output dims |
|--------------------|------------------------|--------------------|---------------------------------|
| Channel embedding  | $W(z x,y)$ lookup      | $z\_encoder$ MLP   | $1 \rightarrow d$               |
| CalcLeft (f-node)  | Circular convolution   | MLP                | $3d \rightarrow d$              |
| CalcRight (g-node) | Normalized product     | MLP                | $3d \rightarrow d$              |
| CalcParent         | Inv. circ. conv.       | Gated residual MLP | $2d \rightarrow d$              |
| Decision           | Probability extraction | MLP                | $d \rightarrow 4$               |
| Re-embedding       | Probability embedding  | MLP                | $4 \rightarrow d$               |

### 5.2 Architecture Details

**Default hyperparameters:**  $d=16$  (embedding dimension),  $hidden=64$ , 2-layer MLPs. Total:  $\sim 39,000$  parameters.

**$z\_encoder$**  (continuous channel  $\rightarrow$  embedding):

`Linear(1, 32)  $\rightarrow$  ELU  $\rightarrow$  Linear(32, d)`

Maps each channel output  $z$  (a scalar float) to a  $d$ -dimensional embedding independently per position. After embedding, bit-reversal permutation is applied.

**CalcLeft / CalcRight MLPs:**

Input: `concat(parent_first_half, parent_second_half, sibling) = 3d dims`  
`Linear(3d, 64)  $\rightarrow$  ELU  $\rightarrow$  Linear(64, 64)  $\rightarrow$  ELU  $\rightarrow$  Linear(64, d)`

The parent edge embedding has size  $2d$ , split into first half (from CalcParent) and second half (linear transform of right child).

**CalcParent (Gated Residual):**

```
gate = sigmoid(Linear(2d  $\rightarrow$  64)  $\rightarrow$  ELU  $\rightarrow$  Linear(64  $\rightarrow$  d))
candidate = MLP(2d  $\rightarrow$  64  $\rightarrow$  64  $\rightarrow$  d)
residual = (left + right) / 2
output = gate  $\times$  candidate + (1 - gate)  $\times$  residual
```

The residual connection provides a stable starting point and ensures gradient flow.

**emb2logits:** `MLP(d  $\rightarrow$  64  $\rightarrow$  64  $\rightarrow$  4)`. Maps embedding to 4-class log-probabilities for joint  $(u,v)$  decision: classes are  $(0,0)$ ,  $(0,1)$ ,  $(1,0)$ ,  $(1,1)$ .

### 5.3 Weight Sharing

ALL operations are **weight-shared** across tree positions and depths. The same CalcLeft MLP handles every f-node regardless of whether it’s at the root (processing  $N$  embeddings) or at a leaf parent (processing 1 embedding). This makes the model  $N$ -independent: a model trained at one  $N$  can decode at any  $N$ .

## 5.4 Tree Walk During Inference

The neural decoder follows the exact same sequential tree walk as the analytical decoder: 1. Navigate to target leaf using CalcLeft/CalcRight/CalcParent (with learned MLPs instead of analytical operations) 2. At leaf: combine embeddings, apply emb2logits for 4-class prediction, argmax for hard decision 3. Apply logits2emb to re-embed the decision 4. Move to next leaf

For  $N=256$ , this is  $\sim 1500$  sequential MLP calls. Each call processes tiny tensors ( $\text{batch} \times d=16$ ), making it CPU-bound.

---

## 6. Training Methodology

### 6.1 Sequential Teacher-Forced Training

The model processes one codeword at a time through the full SC tree walk: - **Forward**: Sequential tree walk, collecting logits at each non-frozen leaf - **Loss**: Cross-entropy between predicted 4-class logits and true (u,v) targets - **Teacher forcing**: During training, TRUE bit decisions are fed at each leaf (not the model’s own predictions) - **Backward**: Gradients flow through the ENTIRE sequential chain

**Gradient depth**:  $O(N \log N)$ . At  $N=256$ , gradients must flow through  $\sim 1500$  sequential operations. This is the fundamental training bottleneck.

### 6.2 Curriculum Learning

Training from scratch fails at  $N \geq 32$  (loss stays at random). The solution: 1. Train at  $N=16$  until convergence 2. Load weights, fine-tune at  $N=32$  3. Continue to  $N=64, 128, 256, \dots$

Each stage inherits the weight-shared tree operations from the previous stage. The operations must generalize across tree depths — this is where the approach eventually fails at large  $N$ .

### 6.3 Learning Rate Schedule

**Critical finding**: The original 48-hour training used cosine annealing with warm restarts, which periodically disrupted learning by jumping the LR back to its initial value. Switching to **stable cosine decay** (single decay, no restarts) significantly improved results: -  $N=128$ :  $0.027 \rightarrow 0.019 \rightarrow 0.017$  BLER -  $N=256$ :  $0.033 \rightarrow 0.019 \rightarrow 0.015$  BLER

### 6.4 Training Time Requirements

---

| N   | Iters needed | Wall time | Best BLER |
|-----|--------------|-----------|-----------|
| 32  | 15K          | 20 min    | 0.046     |
| 64  | 80K          | 12 hr     | 0.026     |
| 128 | 135K         | 28 hr     | 0.017     |
| 256 | 100K         | 16 hr     | 0.015     |
| 512 | 45K+         | 28+ hr    | 0.012     |

---

## 7. Detailed Results

### 7.1 Complete BLER Comparison (GMAC, SNR=6dB, Class B)

| N   | SC    | SCL(4) | SCL(32) | NN-SC | NN-SCL(4) | NN-CA-SCL(4) |
|-----|-------|--------|---------|-------|-----------|--------------|
| 32  | 0.046 | 0.023  | 0.023   | 0.046 | 0.033     | —            |
| 64  | 0.025 | 0.010  | 0.010   | 0.026 | 0.018     | —            |
| 128 | 0.016 | 0.004  | 0.004   | 0.017 | 0.014     | <b>0.002</b> |
| 256 | 0.005 | 0.001  | 0.0003  | 0.015 | 0.021     | 0.022        |
| 512 | 0.001 | 0.000  | 0.000   | 0.012 | —         | —            |

### 7.2 Key Findings

1. **NN-SC matches SC at  $N \leq 128$**  (within 4%). This is the main positive result.
2. **NN-SC hits a  $\sim 0.015$  BLER ceiling** regardless of  $N$ . SC BLER drops with  $N$  (0.016 $\rightarrow$ 0.005 $\rightarrow$ 0.001) but NN stays flat. The model has a fixed per-codeword error rate.
3. **NN-SCL helps at  $N \leq 128$  but hurts at  $N \geq 256$** . At  $N=256$ , the NN’s probability estimates are miscalibrated — list decoding trusts bad confidence scores and prunes the correct path. With  $L=32$  at  $N=256$ , BLER is 4x worse than greedy.
4. **CRC-aided NN-SCL is excellent at  $N=128$**  (BLER=0.002, beating analytical SCL), but fails at  $N=256$  because the correct path gets pruned before CRC can save it.
5. **Analytical SCL( $L=32$ ) is essentially perfect** at  $N=256$  (BLER=0.0003). The gap between NN and analytical grows with  $N$ .

### 7.3 Per-Position Error Analysis ( $N=256$ )

Comparing NN decisions against TRUE bits (5000 codewords): - NN makes 0.67 errors per codeword on average (BER=0.0027) - SC makes 0.04 errors per codeword (BER=0.0001) - Errors are NOT concentrated at specific positions — spread uniformly across all info positions - User U has more errors than V (expected for Class B where U is decoded with less side info) - Later positions in the decoding order have slightly higher error rates

---

## 8. Approaches That Worked

### 8.1 Curriculum Learning (Essential)

Training from scratch fails at  $N \geq 32$ . Curriculum  $N=16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$  is required.

### 8.2 Stable Cosine LR (Critical)

Switching from warm restarts to single cosine decay improved  $N=128$  from 1.69x to 1.04x SC.

### 8.3 Freeze & Extend (Breakthrough at N=128)

Freeze shared CalcLeft/CalcRight at levels 1-5 (proven good for N=64), add NEW trainable level-6 specific MLPs. Only 16K new trainable params. Reached 1.04x SC at N=128 in 2 hours (vs 12 hours for continued training).

**Why it works:** Different tree levels need different transformations. The shared model compromises across levels. Level-specific MLPs can specialize.

**Why it fails at N=256:** Training only the deepest level doesn't fix accumulated errors from multiple levels. Multi-level unfreeze (levels 5+6+7) was tried but only reached 0.030 (6x SC) after overnight training.

### 8.4 Scheduled Sampling (Modest Improvement)

During training, sometimes feed the model's OWN prediction instead of the true bit (probability ramps from 0% to 30%). Improved N=256 from 0.019 to 0.015 (21% gain, 5000-cw validated).

### 8.5 CRC-Aided Neural SCL (Excellent at N=128)

Concatenate CRC-8 to User U's message. After SCL produces L candidates, check each path's CRC. Pick the one that passes. At N=128: BLER=0.002 (5x better than plain SCL).

### 8.6 C++ Extension (1.34x Training Speedup)

Custom PyTorch C++ extension for the tree walk forward pass. Eliminates Python interpreter overhead at each of the 1500 MLP calls. Verified identical output to Python.

---

## 9. Approaches That Failed

### 9.1 Larger Model (d=32, 157K params)

6.3x more parameters. Matched SC at N=32 but failed at N>=64 due to insufficient training budget (30K iters vs needed 200K+). Curriculum transfer to N=128 failed completely (BLER=1.0). **Conclusion:** More capacity doesn't help without proportionally more training.

### 9.2 NPD-Style Fast-CE (Parallel Teacher-Forced Training)

Inspired by Aharoni et al.'s Neural Polar Decoder. Processes all N positions at each tree depth simultaneously, reducing gradient depth from  $O(N \log N)$  to  $O(\log N)$ .

**Why it works for NPD (single-user):** Binary output with sign-flip BitNode (output =  $MLP + e_{\text{odd}} \times u_{\text{sign}} + e_{\text{even}}$ ) matches the analytical g-node formula exactly. The skip connection provides the correct inductive bias.

**Why it fails for MAC:** The 4-class joint (u,v) structure doesn't decompose through binary sign flips. We tried: - 2-group sign encoding (half dims for u, half for v): loss plateaus at 0.30 (near random 1.39) - WHT 4-group sign encoding: loss plateaus at 0.29 - One-hot decision embedding + residual: loss plateaus at 0.30

**All variants converge to the same ~0.30 loss plateau.** The MAC's 2x2 circular convolution doesn't factor into the element-wise operations that NPD uses.

### 9.3 Two-Decoder Binary Approach

Train two independent single-user NPD decoders: one for U (marginal channel), one for V (conditional channel). Each is binary, matching NPD’s proven architecture.

**Why it fails:** At Class B symmetric rates,  $R_u = 0.469 > I(Z;X) = 0.464$ . User U operates ABOVE its marginal capacity — it cannot be decoded alone. The interleaved path is information-theoretically necessary.

### 9.4 Residual Connections in CalcLeft/CalcRight

Add skip connection:  $\text{output} = \text{MLP}(\text{input}) + \text{parent\_first\_half}$ . Inspired by NPD’s BitNode residual.

**From scratch:** BLER=1.0 — the skip connection dominates at initialization, MLP has no gradient signal. Never learns.

**Fine-tuning existing model:** BLER=1.0 after 5K iters — the MLP was trained to output the FULL answer; adding +p\_first doubles the parent component. Would need 50K+ iters to adapt.

**Why NPD’s residual works but ours doesn’t:** NPD’s skip ( $e_{\text{odd}} \times u_{\text{sign}} + e_{\text{even}}$ ) IS the analytical formula — the MLP starts from a perfect decoder. Our skip (+p\_first) doesn’t match any analytical formula.

### 9.5 Snapshot Training (Operation-Level Distillation)

Run the analytical SC decoder, record the exact  $2 \times 2$  probability tensor at every edge during the tree walk. Train each NN operation independently against these analytical targets.

**Standalone (from scratch):** Operations learn individually (per-operation MSE drops to 0.02) but BLER=1.0 when chained. Errors compound over 500+ sequential operations.

**Probability domain variant:** Interface all operations via probability vectors (not embeddings). Works at  $N=8$  (matches SC!) but degrades at  $N \geq 128$  due to information loss in the  $\text{prob} \rightarrow \text{emb} \rightarrow \text{MLP} \rightarrow \text{emb} \rightarrow \text{prob}$  conversion chain. Loss plateaus at  $\sim 1.1$  regardless of training budget.

**Combined with end-to-end:** Added snapshot MSE as auxiliary loss during sequential training. Snapshot loss pushes CalcLeft/CalcRight to expect analytical tensor inputs, but real decoding feeds embeddings from z\_encoder/CalcParent. Destroys the model (BLER=1.0).

### 9.6 Multi-Depth Auxiliary Loss

Compute CE loss at intermediate tree edges, not just leaves. Applied to CalcLeft/CalcRight outputs and CalcParent outputs.

**CalcLeft/CalcRight targets:** XOR-decomposed codeword bits. But these targets are WRONG — the SC decoder’s intermediate tensors don’t match simple XOR of codeword bits (they depend on the full decoding history).

**CalcParent targets:** XOR of children’s MESSAGE bits. These targets are CORRECT (validated at  $N=8,16,32$ ).  $\text{circ\_conv}(\text{delta\_}(a,b), \text{delta\_}(c,d)) = \text{delta\_}(a \text{ XOR } c, b \text{ XOR } d)$ .

**Results:** Even with correct CalcParent targets, any significant auxiliary loss weight prevents the leaf CE from dropping. At  $\lambda=0.01$ : leaf loss stuck at 0.83 (from scratch). At equal weight: leaf loss stuck at 0.83 for 9K iters, then slowly recovered but 20x slower than baseline.

**Multi-depth from scratch:** Helps at  $N=128$  (0.079 vs baseline 0.094, 16% improvement) but 3.2x slower. Doesn't change the  $N=256$  picture.

**Why NPD's multi-depth works but ours doesn't:** NPD trains with multi-depth from the start — the embeddings LEARN to be decodable at every depth. Our embeddings evolved to be information carriers, not directly decodable.

## 9.7 Per-Level CalcLeft/CalcRight

Separate MLPs per tree level instead of weight-sharing (189K params for 10 levels).

**Key finding:** Solves curriculum transfer to  $N=128$  (starts at BLER=0.265 vs shared model's 1.0). Confirms that different tree levels need different transformations.

**Result:** BLER=0.056 at  $N=128$  after 10 hours — worse than shared model (0.017) due to slow convergence with 189K params.

---

## 10. Analysis: Why the Gap at $N \geq 256$

### 10.1 The ~0.015 BLER Ceiling

The  $d=16$  model converges to approximately 0.015 BLER regardless of  $N$ : -  $N=128$ : 0.017 -  $N=256$ : 0.015 -  $N=512$ : 0.012

SC BLER drops rapidly with  $N$  (0.016→0.005→0.001). The NN doesn't benefit from increasing code length — it has a fixed per-codeword accuracy limit.

### 10.2 Per-Position Error Rate

At  $N=256$ , the NN disagrees with the TRUE bits on ~0.27% of info positions (0.67 errors per codeword). This 0.27% error rate needs to drop to ~0.04% to match SC. The errors are spread uniformly across positions — no specific positions are responsible.

### 10.3 Signal Range Through Tree Levels

During the ACTUAL tree walk at  $N=256$ , embedding statistics by level: - CalcLeft output std: 3.7-4.9 (stable) - CalcRight output std: 6.1-8.1 (stable) - CalcParent output std: 7.4-11.0 (stable/growing)

**The signal does NOT collapse** during the real tree walk (CalcParent re-injects decided information). The earlier observation of range collapse was from an artificial leftmost-CalcLeft-only chain, not the actual tree walk.

### 10.4 Visit Pattern

At  $N=256$ , each tree node is visited ~6 times. Level 7 (leaf parents) accounts for half of all operations (768 out of 1517). The total of 1517 MLP calls per forward pass is the training speed bottleneck.

## 10.5 Root Cause Assessment

The fundamental issue is NOT: - Signal range collapse (stable in real tree walk) - Specific failing tree levels (errors are uniform) - Model capacity (d=32 doesn't help) - Training tricks (all plateau at same BLER)

The fundamental issue IS: - **Each MLP call introduces ~0.3% approximation error** on the tensor operations - Over 1500 calls, these small errors accumulate into 1-2 wrong bit decisions per codeword - No training method can push the per-operation accuracy below this threshold with d=16 embeddings - The  $O(N \log N)$  gradient depth makes it impossible to train the fine-grained corrections needed

---

## 11. Comparison with NPD (Single-User Neural Decoder)

The NPD (Aharoni et al.) achieves near-SC performance at  $N=1024$  for single-user channels. Key architectural differences:

| Feature           | NPD                                          | Our MAC Decoder                      |
|-------------------|----------------------------------------------|--------------------------------------|
| Output            | Binary (1 bit)                               | 4-class joint (u,v)                  |
| Training          | Parallel<br>fast_ce, $O(\log N)$ depth       | Sequential, $O(N \log N)$ depth      |
| BitNode           | sign flip + residual<br>(matches analytical) | No natural decomposition for 4-class |
| Parameters        | ~11K (d=8)                                   | ~39K (d=16)                          |
| Works at $N=1024$ | Yes                                          | No (BLER plateau)                    |

**Why NPD scales and we don't:** NPD's parallel training (fast\_ce) gives  $O(\log N)$  gradient depth — only 10 sequential steps for  $N=1024$ . Our sequential training has  $O(N \log N) = \sim 10,000$  steps. NPD's BitNode has a natural sign-flip structure matching the analytical g-node; our MAC joint structure has no such decomposition.

---

## 12. Codebase Overview

### Core Library (polar/)

- `encoder.py` —  $O(N \log N)$  polar encoder with batch support
- `channels.py` — BEMAC, ABNMAC, GaussianMAC
- `design.py` — Bhattacharyya + GA density evolution + GMAC design
- `design_mc.py` — Monte Carlo genie-aided design
- `decoder.py` — Unified SC decoder (auto-dispatch)
- `decoder_scl.py` — SC List decoder
- `decoder_interleaved.py` —  $O(N \log N)$  for all monotone chain paths
- `eval.py` — BER/BLER evaluation pipeline

## Neural Decoders (neural/)

- `ncg_pure_neural.py` — Weight-shared SC decoder (39K params)
- `ncg_gmac.py` — GMAC variant with continuous `z_encoder`
- `neural_scl.py` — Neural SCL decoder (list decoding on NN)
- `csrc/fast_tree_walk.cpp` — C++ extension (1.34x speedup)

## Pre-computed Resources

- `designs/` — MC frozen sets for  $N=8-2048$ , SNR 0-10 dB
  - `saved_models/` — Trained checkpoints for all  $N$
- 

## 13. Open Problems and Future Directions

### 13.1 Closing the $N \geq 256$ Gap

The most promising unexplored direction: **larger  $d$  (32 or 64) with sufficient training time (days, not hours)**. The  $d=32$  model was never given enough training — it needs 200K+ iters per  $N$ , which requires days of compute on CPU.

### 13.2 Parallel Training for MAC

Adapting NPD’s `fast_ce` to the 4-class MAC joint structure remains the highest-impact unsolved problem. The  $2 \times 2$  circular convolution doesn’t decompose through sign flips. A correct decomposition (perhaps via Walsh-Hadamard transform or group-theoretic approach) would enable  $O(\log N)$  gradient depth.

### 13.3 CRC-Aided Neural SCL

CRC-aided NN-SCL at  $N=128$  achieves BLER=0.002 (beating analytical SCL). Extending this to  $N=256$  requires better probability calibration so the correct path survives in the  $L$  candidates.

### 13.4 Channels with Memory

The NPD paper demonstrates neural decoding of channels with memory (ISI, channels with state) using DINE/MINE for channel embedding. Adapting this to the MAC setting would extend the project to practical scenarios.

---

## 14. References

1. E. Arıkan, “Channel polarization: A method for constructing capacity-achieving codes,” *IEEE Trans. Inf. Theory*, 2009.
2. S. B. Onay, “Successive cancellation decoding of polar codes for the two-user MAC,” *IEEE ISIT*, 2013.
3. Y. Ren, Z. Li, P. M. Olmos, “SC decoding of polar codes for the two-user MAC using computational graphs,” in preparation, 2025.
4. S. Aharoni, R. Misoczki, E. Ordentlich, “Neural polar decoders for 5G,” *IEEE JSAC*, 2024.
5. I. Tal, A. Vardy, “List decoding of polar codes,” *IEEE Trans. Inf. Theory*, 2015.

---

## Appendix A: Key Hyperparameters

| Parameter         | Value                      |
|-------------------|----------------------------|
| Embedding dim d   | 16                         |
| Hidden dim        | 64                         |
| MLP layers        | 2                          |
| z_encoder hidden  | 32                         |
| Learning rate     | 5e-5 to 3e-4               |
| Optimizer         | AdamW (weight_decay=1e-5)  |
| Gradient clipping | 1.0                        |
| LR schedule       | Cosine decay (no restarts) |
| Batch size        | 4-64 (depends on N)        |

## Appendix B: Validated Mathematical Properties

1. **CalcParent of two delta tensors:**  $\text{circ\_conv}(\text{delta\_}(a,b), \text{delta\_}(c,d)) = \text{delta\_}(a \oplus c, b \oplus d)$ 
  - CalcParent output = XOR of children’s message bits (validated at N=8,16,32)
2. **After last CalcParent visit**, every edge tensor is a clean delta function — one class has probability 1.0, rest are 0.
3. **GA design != MC design for Class B:** GA assumes extreme path, MC accounts for interleaved path. Using wrong design gives 70%+ BLER.

## Appendix C: Training Speedups Explored

| Approach                        | Speedup          | Notes                                |
|---------------------------------|------------------|--------------------------------------|
| Pre-computed operation schedule | 1.02x            | Navigation overhead was minimal      |
| C++ forward pass extension      | 1.34x            | Eliminates Python dispatch           |
| torch.compile                   | 0.76x (slower)   | Overhead dominates for tiny tensors  |
| MPS (Apple GPU)                 | 0.2x (5x slower) | CPU→GPU transfer dominates           |
| Multiprocessing                 | Not viable       | PyTorch autograd + fork incompatible |
| Larger batch                    | More samples/hr  | Each iter slower but more data       |
| Batching independent ops        | Not possible     | SC decoding is 100% sequential       |